

# 8-Point Pilot State Probe

Read-only investigation requested against the production database and API. **Production credentials are not available to this assessor.** All probes below were run against the only database reachable from the source tree: localhost:5433/ascure (.env DATABASE\_URL), which contains the “demo-tenant — Demo Utility Tenant” dataset. The same exact Prisma queries are included verbatim in §10 so an operator with production access can re-run them and compare. No code modified.

**SCOPE NOTE — Results in §1–§8 reflect the connected DB (dev/demo). They are not a claim about production. Treat them as the same-shape answers the same code would return if pointed at production. Where the user’s prior turn established that production state diverges (the GET /enterprise/options response missing 5 codes), production may also diverge here. The §10 queries are the way to confirm.**

## 1. SiteVisit count by tenant

| tenantId                             | Tenant label                      | SiteVisit count |
|--------------------------------------|-----------------------------------|-----------------|
| 1ef638af-bf76-472c-9daa-299d6edd5c8a | demo-tenant — Demo Utility Tenant | 4               |

Connected DB has SiteVisits in exactly one tenant.

## 2. Last 20 SiteVisit rows

All 4 rows shown; no more exist. All are status=ACTIVE and were created by the same TECHNICIAN user (53ee3bcf-...).

| id (truncated) | status / validation | mainheadId   | createdBy UserId | startedAt            | complete dAt |
|----------------|---------------------|--------------|------------------|----------------------|--------------|
| 61eaa849-...   | ACTIVE / PENDING    | 31518197-... | 53ee3bcf-...     | 2026-05-29 10:03:20Z | —            |
| 339d5da0-...   | ACTIVE / PENDING    | null         | 53ee3bcf-...     | 2026-05-29 04:23:12Z | —            |
| 6f77fd90-...   | ACTIVE / PENDING    | null         | 53ee3bcf-...     | 2026-05-29 04:19:32Z | —            |
| 7a4e5686-...   | ACTIVE / null       | null         | 53ee3bcf-...     | 2026-04-23 13:35:41Z | —            |

Three of the four visits have mainheadId=null — they were created before the G2 DTO change required mainheadId. Only the newest one carries a structured MAINHEAD. None are completed.

## 3. Last 20 Asset rows (showing createdDuringVisitId)

All 2 rows shown; no more exist. **Both rows have** createdDuringVisitId: null — neither asset was linked to a SiteVisit at create-time.

| id (truncated) | substationId | assetCode                   | createdDuringVisitId | createdAt            |
|----------------|--------------|-----------------------------|----------------------|----------------------|
| bd03324c-...   | da18f394-... | R4E-20260529100318-ASSET-02 | null                 | 2026-05-29 10:03:19Z |
| 98c3018b-...   | da18f394-... | SAVR-001                    | null                 | 2026-04-23 13:29:37Z |

Summary: **with createdDuringVisitId: 0; without: 2.** Mobile's AddAssetScreen passes `createdDuringVisitId: siteVisitId`, so either `siteVisitId` was undefined at submit time, or the assets predate the visit-linkage code path. Cross-reference with Probe 4.

## 4. Last 20 SiteVisitAsset link rows

3 link rows exist. Every link was created with `source= 'INSPECTION'` — meaning these links were materialised by the inspection submission pathway (`materializeImplicitVisitAssetLinks`) and NOT by the create-asset path. This is consistent with Probe 3: assets persisted without a visit link, then later got linked when an inspection was submitted against them.

| siteVisitId  | assetId      | source     | addedAt              |
|--------------|--------------|------------|----------------------|
| 61eaa849-... | bd03324c-... | INSPECTION | 2026-05-29 10:03:20Z |
| 61eaa849-... | 98c3018b-... | INSPECTION | 2026-05-29 10:03:20Z |
| 339d5da0-... | 98c3018b-... | INSPECTION | 2026-05-29 04:23:12Z |

## 5. Admin tenantId(s)

| id           | email                 | tenantId     | organizationId | isActive |
|--------------|-----------------------|--------------|----------------|----------|
| 6ca97a0b-... | admin@ascure.local    | 1ef638af-... | 4bfcfdda-...   | true     |
| 8d9aea1d-... | g1.admin@ascure.local | 1ef638af-... | 4bfcfdda-...   | true     |

Both ADMIN users are in tenant 1ef638af-... — same as the four SiteVisit rows. On the connected DB, an ADMIN list query would return all 4 visits.

## 6. Pilot user tenantId(s)

TECHNICIAN-role users (the canonical pilot-user role):

| email                   | tenantId     | teamId       | mainheadId   |
|-------------------------|--------------|--------------|--------------|
| g1.user.c@ascure.local  | 1ef638af-... | null         | null         |
| technician@ascure.local | 1ef638af-... | 9d4bea53-... | 31518197-... |

Both TECHNICIANS are in the same tenant as the ADMIN users. The pilot user that created all 4 SiteVisits is technician@ascure.local (53ee3bcf-...); the other technician (g1.user.c) is not linked to a team or MAINHEAD and has never created a visit on this DB.

**Tenant alignment check: PASS on connected DB.** Both ADMIN and pilot users share tenant 1ef638af-...; an ADMIN list query is not gated out by tenant scoping.

## 7. Did POST /site-visits succeed during the pilot run?

**Cannot be answered definitively from the database alone.** The HTTP access log or API audit log is the source of truth for endpoint outcomes. A reasonable proxy is “did a SiteVisit row get persisted for this user around the pilot timeframe?”

On the connected DB, the answer to the proxy question is **yes**: every one of the 4 SiteVisit rows was created by technician@ascure.local. The most recent is 2026-05-29 10:03:20Z, status ACTIVE, validationStatus PENDING.

- If production has any SiteVisit row with createdByUserId = (pilot user id) and createdAt inside the pilot window, the create call succeeded at least once.
- If production has zero such rows, the create call did not succeed for that user during that window. Check the API log for POST /site-visits 4xx responses around the pilot run.
- Common 4xx causes after Governance G2: missing mainheadId (DTO-level required), team-membership rejection (non-admin user not in dto.teamId), missing GPS for new-Pencawang check-in.

## 8. SiteVisit rows for the pilot user's tenant

| tenantId     | Tenant label                      | SiteVisit count | Oldest               | Newest               |
|--------------|-----------------------------------|-----------------|----------------------|----------------------|
| 1ef638af-... | demo-tenant — Demo Utility Tenant | 4               | 2026-04-23 13:35:41Z | 2026-05-29 10:03:20Z |

**Answer (connected DB): yes — 4 rows exist for the pilot user's tenant.** Therefore the “admin sees zero” symptom cannot be reproduced on this DB. If production shows zero rows for the same tenant, the

gap is environmental (the visit create call never persisted a row on production).

## 9. Cross-checks against the symptom

| Symptom (reported)                    | Connected DB                                                                         | Interpretation                                                                                                                                                                               |
|---------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Admin Assets page shows new assets    | 2 Asset rows visible; both have createdDuringVisitId=null                            | Consistent. Assets are listed via Substation, not via Visit. Null linkage does not hide them from the Assets page.                                                                           |
| Admin Site Visits page shows 0 visits | 4 SiteVisit rows exist; tenant matches Admin                                         | Does NOT reproduce on connected DB. An ADMIN list call would return 4 rows. Production-only divergence — needs probe via §10.                                                                |
| Operations / Sessions shows 0         | 7 OperationalSession rows exist on connected DB but mobile pilot does not write them | Always expected for mobile-only pilot flow. The mobile create path does not produce OperationalSession rows; sessions seen on dev were created via a separate path.                          |
| Dashboard shows 0 visits              | Counts use the same accessible-where as the list; would show 4 on connected DB       | Same conclusion as Site Visits page — no reproduction here; needs production probe.                                                                                                          |
| Operations Board shows 0 defects      | No defects in dataset                                                                | Consistent. Defects require inspections; the 4 inspections in the dataset are PENDING and have not produced defects in this dataset. On production this is downstream of the visit question. |

## 10. Re-run instructions for production

Point DATABASE\_URL at the production DB and run the same script. Read-only; no writes anywhere. Save the output and compare against the §1–§8 sections of this report.

```
// PROBE 1 – SiteVisit count by tenant
prisma.siteVisit.groupBy({ by: ['tenantId'], _count: { _all: true } })

// PROBE 2 – Last 20 SiteVisit rows
prisma.siteVisit.findMany({
  orderBy: { createdAt: 'desc' }, take: 20,
  select: { id, tenantId, teamId, substationId, createdByUserId,
    mainheadId, status, validationStatus, startedAt,
    completedAt, createdAt }
})

// PROBE 3 – Last 20 Asset rows
prisma.asset.findMany({
  orderBy: { createdAt: 'desc' }, take: 20,
  select: { id, tenantId, substationId, assetCode,
    createdDuringVisitId, createdByUserId, createdAt }
})

// PROBE 4 – Last 20 SiteVisitAsset link rows
prisma.siteVisitAsset.findMany({
  orderBy: { addedAt: 'desc' }, take: 20,
  select: { siteVisitId, assetId, addedByUserId, source, addedAt }
})

// PROBE 5 – Admin users
prisma.user.findMany({
  where: { role: 'ADMIN' },
  select: { id, email, tenantId, organizationId, isActive },
```

```

}))

// PROBE 6 – Pilot users (TECHNICIAN role; substitute pilot email if known)
prisma.user.findMany({
  where: { role: 'TECHNICIAN' },
  select: { id, email, tenantId, organizationId, teamId,
    mainheadId, isActive },
})

// PROBE 7 – SiteVisits authored by pilot user during pilot window
prisma.siteVisit.findMany({
  where: { createdByUserId: { in: [ /* pilot ids */ ] },
    createdAt: { gte: new Date('PILOT_START'), lte: new Date('PILOT_END') } },
  orderBy: { createdAt: 'desc' }, take: 20,
  select: { id, tenantId, status, createdAt },
})

// PROBE 8 – SiteVisit count for the pilot user's tenant
prisma.siteVisit.count({ where: { tenantId: 'PILOT_TENANT_ID' } })

```

Decision tree after running on production:

- **Probe 1 / 8 returns 0 for the pilot tenant:** no visits were ever persisted. Inspect API log for POST /site-visits 4xx responses; the most likely rejection after G2 is missing/invalid mainheadId at the DTO level (create-site-visit.dto.ts:46).
- **Probe 5 admin.tenantId ≠ Probe 8 pilot tenant with rows:** rows exist but in a different tenant than admin. Fix tenant configuration; existing rows would need a tenant-migration script before they become visible to admin.
- **Probe 3 has rows with createdDuringVisitId=null: matches the dev signature.** Mobile is persisting assets through the substation path without the visit linkage — exactly what we see on dev. Each such asset is the orphaned tail of a visit-create that did not complete cleanly.
- **Probe 4 shows rows with source='INSPECTION':** downstream materializeImplicitVisitAssetLinks is stitching the asset-visit links at inspection time. Useful but does not retroactively fix the missing inspection-and-defect chain if the visit never was created.